RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

QUEUES

Types, Implementation and Applications

Simple • Circular • Double-Ended • Priority

Arrays and Linked Lists

Scheduling • Buffering • Simulation Problems

Learning Objectives

Explain the four queue types: simple, circular, double-ended and priority queues.

Describe array and linked-list representations of queues.

Trace insertion and deletion in simple and circular queues.

Explain how a deque and priority queue differ from a standard FIFO queue.

Identify queue applications in scheduling, buffering and simulation problems.

Queue Types: At a Glance

Simple Queue

FIFO; insertion at rear and deletion at front.

Circular Queue

The array is treated as circular to reuse free positions.

Double-Ended Queue (Deque)

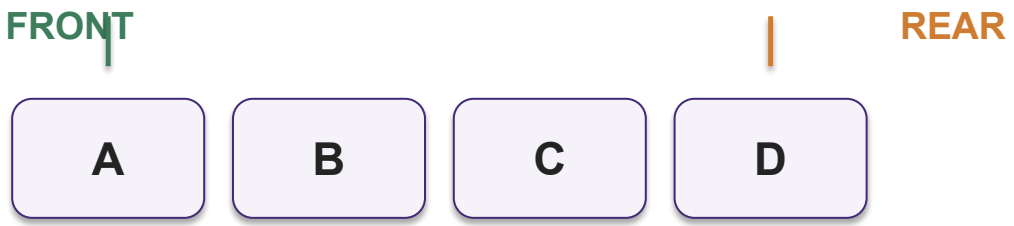
Insertion and deletion are permitted at either end.

Priority Queue

The next item is selected according to priority.

Simple Queue

A simple queue is a FIFO list: the first inserted item is the first deleted.



Insert

Add an item at the rear.

Delete

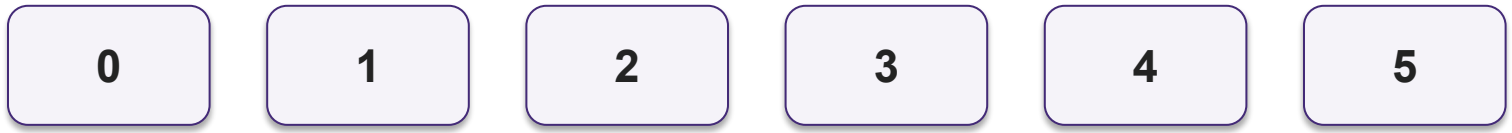
Remove an item from the front.

Order

Arrival order determines service order.

Circular Queue

A circular queue avoids the wasted-space problem of a simple linear array queue.



After index 5, the next position can wrap to index 0.

next = (current + 1) mod MAX_QUEUE_SIZE

Benefit

Reuses positions released by deletions.

Need

Careful empty/full conditions are required.

Use

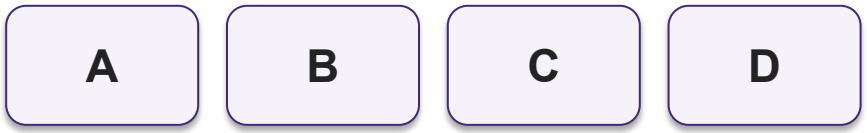
Useful for fixed-size buffers.

Double-Ended Queue (Deque)

A deque is a linear list in which additions and deletions may be made at either end.

INSERT / DELETE

INSERT / DELETE



Input-Restricted Deque

Insertion is allowed at one end; deletion is allowed at both ends.

Output-Restricted Deque

Deletion is allowed at one end; insertion is allowed at both ends.

Priority Queue

A priority queue selects the next element by priority rather than by simple FIFO order.

Job A	Job B	Job C	Job D
Priority 3	Priority 1	Priority 2	Priority 1

If lower number means higher priority, Job B and Job D are served before Job C and Job A.

Service

Priority determines the next item.

Tie

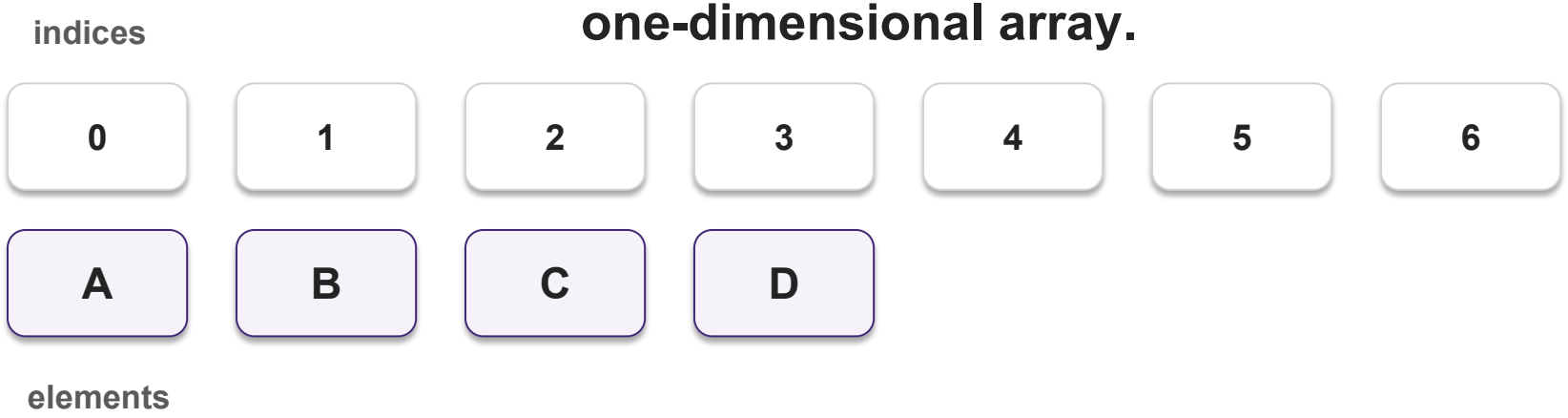
Equal priorities can be handled FIFO.

Use

Scheduling and service systems.

Array Representation of a Queue

In a sequential representation, the queue elements occupy positions in a one-dimensional array.



front identifies the next element to be deleted.

rear identifies the end where the next element is inserted.

Simple Queue: Array Implementation

```
Queue {
    KeyType queue[MAX];
    int front;
    int rear;
};

Add(item):
    check full
    rear++
    queue[rear] = item

Delete():
    check empty
    item = queue[front]
    front++
```

Array storage

Fixed-capacity contiguous storage.

Enqueue

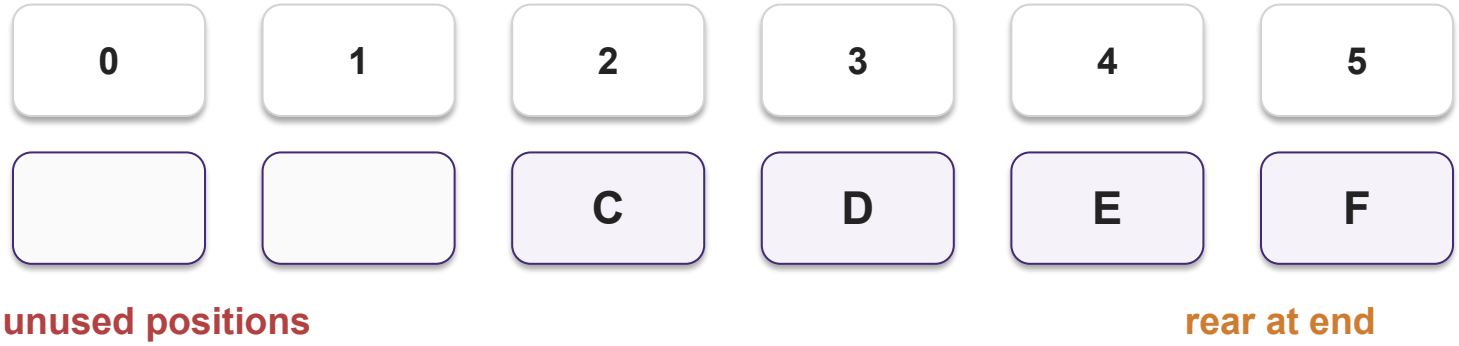
Advance rear and store the item.

Dequeue

Read front item and advance front.

Linear Queue: Space Utilization Problem

After deletions, free cells may remain at the beginning while rear reaches the end.



The simple array representation can require movement of elements to reclaim space.

The worst-case cost of shifting can be O(n).

The circular queue eliminates repeated shifting by wrapping around.

Circular Queue: Array Implementation

```
rear = (rear + 1) % MAX_QUEUE_SIZE;
front = (front + 1) % MAX_QUEUE_SIZE;

Enqueue:
    move rear circularly
    store item at rear

Dequeue:
    move front circularly
    remove item at front
```

Wrap-around

The position after the last index is the first index.

Advantage

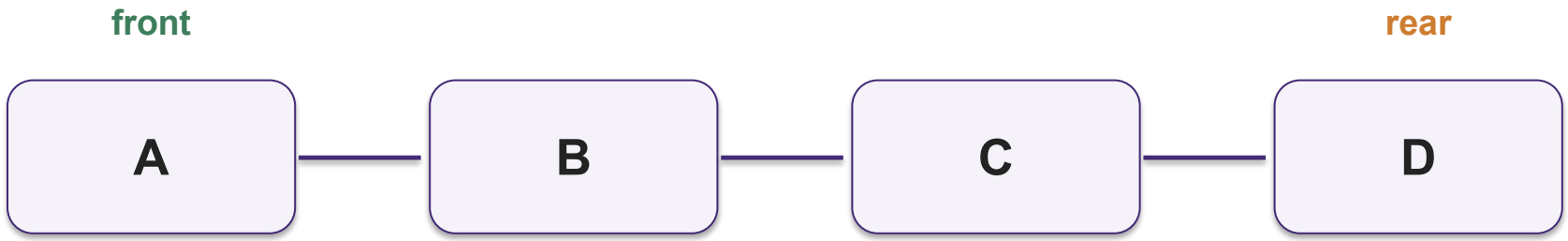
Free positions are reused without shifting.

Condition

Full and empty states must be distinguished.

Linked-List Representation

A linked queue stores elements in dynamically allocated nodes connected by links.



Node

Contains data and a link to the next node.

Front pointer

Points to the first node.

Rear pointer

Points to the last node.

Linked Queue: Enqueue

```
new = allocate node;
new->data = item;
new->link = NULL;

if (rear == NULL)
    front = rear = new;
else {
    rear->link = new;
    rear = new;
}
```

Empty queue

The first node becomes both front and rear.

Non-empty queue

Link the new node after rear, then move rear.

Cost

With a rear pointer, insertion is O(1).

Linked Queue: Dequeue

```
if (front == NULL)
    QueueEmpty();
else {
    temp = front;
    item = temp->data;
    front = front->link;
    free(temp);

    if (front == NULL)
        rear = NULL;
}
```

Empty queue

Report the queue-empty condition.

Normal case

Remove the front node and advance front.

Last node

If front becomes NULL, rear also becomes NULL.

Deque Implementation

Using an Array

A circular array is suitable for a deque.

Maintain front and rear indices.

Operations:
- insertFront
- insertRear
- deleteFront
- deleteRear

Using Linked Nodes

A doubly linked list is convenient because each node has links in both directions.

Both ends can be updated efficiently.

Why Use a Deque?

Choose a deque when an application needs access at both ends rather than strict one-way FIFO access.

Priority Queue Implementation

Array / List

An unordered representation makes insertion simple but may require searching for the next highest-priority item.

An ordered representation makes deletion easier but can increase insertion cost.

Linked List

Nodes can be maintained in sorted or unsorted priority order.

The ordering rule determines how the next item is selected.

Priority Rule

Priority, not merely arrival time, determines service order.

Equal priorities may be processed in FIFO order.

Array vs Linked-List Implementation

Aspect	Array	Linked list
Capacity	Usually fixed	Dynamic
Storage	Contiguous	Non-contiguous nodes
Overflow	When capacity is reached	When memory allocation fails
Space reuse	Circular form reuses cells	Deleted nodes are released
Pointers	Indices: front/rear	Node pointers: front/rear
Typical enqueue/dequeue	O(1) with suitable design	O(1) with front/rear pointers

Applications: Scheduling

Queues are fundamental in systems where jobs or requests wait for service.

CPU / Job Scheduling

Jobs waiting for processor service can be maintained in queues.

Priority scheduling can use a priority queue when urgency or priority matters.

Print Scheduling

Print requests wait for the printer and are processed according to the selected service policy.

Service Requests

Requests arriving faster than they can be handled can wait in a queue until a resource becomes available.

Applications: Buffering

Queues provide temporary storage when data arrives and is processed at different rates.

I/O Buffer

Data can wait in a queue between an input device and a processor or output device.

Network Buffer

Packets may wait in a queue before transmission or processing.

Circular Buffer

A circular queue is useful when a fixed-size storage area must be reused continuously.

Applications: Simulation Problems

Queues model waiting lines and event processes in simulation problems.

Bank / Service Counter

Customers arrive, wait, receive service and leave. A queue models the waiting line.

Traffic / Network

Vehicles or packets can be modeled as entities waiting for service or a resource.

Event Simulation

Events can be maintained according to their processing order or priority.

Simulation questions often ask: Who arrives? Who waits? Who is served next? What is the waiting time?

Applications and Appropriate Queue Type

Application	Suitable queue	Reason
Ordinary waiting line	**Simple queue**	Service follows arrival order.
Fixed-size data buffer	**Circular queue**	Freed storage positions are reused.
Access at both ends	**Deque**	Both ends support insertion/deletion.
Urgent job scheduling	**Priority queue**	Priority determines service order.
Event / service simulation	**Simple or priority queue**	Depends on whether arrival order or event priority controls service.